

Лабораторная работа № 7

Тема: Управление системными службами через систему инициализации systemd, настройка автозагрузки и анализ журналов с помощью journalctl.

Цель работы: Закрепить практические навыки администрирования фоновых компонентов в Linux: научиться управлять жизненным циклом служб (systemctl), контролировать параметры автозагрузки, безопасно модифицировать конфигурации юнитов, применять изоляцию через механизм маскирования (mask), а также проводить комплексную диагностику сбоев с помощью утилиты journalctl.

Необходимое ПО

- Эмулятор терминала для подключения по SSH.
- Развернутый Linux-сервер (Ubuntu/Debian).
- Установленная демонстрационная сетевая служба (например, веб-сервер **Nginx** или служба **SSH / OpenSSH-Server**). В рамках задания шаги будут показаны на примере веб-сервера nginx (если он не установлен, перед началом работы выполните `sudo apt update && sudo apt install nginx -y`).

Ситуация

Вы работаете системным администратором. Руководство поставило перед вами комплексную задачу по обслуживанию и аудиту веб-платформы компании:

1. Проверить текущее состояние веб-сервера и настроить его так, чтобы он гарантированно автоматически запускался при включении или аварийной перезагрузке сервера.
2. Продемонстрировать навыки «горячей» перезагрузки конфигурации без обрыва текущих пользовательских сессий, а также симитировать сбой службы для отработки навыков поиска ошибок в логах.
3. Полностью и гарантированно заблокировать в системе неиспользуемую или потенциально опасную службу, исключив её случайный запуск другими администраторами или скриптами автоматизации.

Требования к выполнению работы

1. **Минимизация простоев:** При изменении настроек веб-сервера отдавать приоритет мягкой перевычитке конфигурации (reload), а не полной жесткой перезагрузке всей службы (restart), где это возможно.
2. **Осознанность автозагрузки:** Помнить, что статус active (running) показывает состояние службы прямо сейчас, но не гарантирует её старт после перезагрузки. Всегда проверять статус автозагрузки отдельно.
3. **Целевой аудит логов:** При поиске ошибок запрещено читать системный лог целиком. Студент должен уметь применять точечную фильтрацию по имени конкретного юнита и временным рамкам.

Порядок выполнения работы

Этап 1. Управление состояниями службы в реальном времени (systemctl status / start / stop)

1. Подключитесь к серверу по SSH под своей учетной записью с правами sudo.
2. Проверьте текущий статус веб-сервера Nginx:

```
sudo systemctl status nginx
```

Изучите вывод терминала. Зафиксируйте, находится ли служба в состоянии active (running) или inactive (dead), а также какой главный системный идентификатор (PID) ей присвоен.

3. Остановите службу командой `sudo systemctl stop nginx`. Повторно проверьте статус, чтобы убедиться, что процессы завершились.
4. Запустите службу обратно командой `sudo systemctl start nginx`.

Этап 2. Управление автозагрузкой (enable / disable / is-enabled)

1. Проверьте, добавлена ли служба в автозагрузку операционной системы при старте сервера:

```
sudo systemctl is-enabled nginx
```

2. Отключите автозагрузку веб-сервера:

```
sudo systemctl disable nginx
```

Обратите внимание на вывод терминала — система удаляет символические ссылки (symlinks) из каталогов инициализации.

3. Включите автозагрузку обратно, сделав её постоянной:

```
sudo systemctl enable nginx
```

Убедитесь, что systemd заново создал необходимые симлинки, связывающие юнит с таргетами запуска.

Этап 3. Мягкая перезагрузка и внесение изменений (reload vs restart)

1. Откройте для редактирования unit-файл конфигурации службы Nginx (или её стандартный конфигурационный файл /etc/nginx/nginx.conf) и добавьте в него комментарий или измените любой незначительный параметр.
2. Согласно правилам systemd из лекции, после изменения конфигурации юнитов необходимо обновить кэш самой системы инициализации. Выполните команду:

```
sudo systemctl daemon-reload
```

3. Продемонстрируйте применение изменений без разрыва сетевых соединений пользователей («горячая» перевычитка параметров):

```
sudo systemctl reload nginx
```

4. Объясните в отчете, в какой ситуации вместо reload администратор обязан применить команду restart.

Этап 4. Блокировка служб через механизм маскирования (mask / unmask)

1. Симулируйте ситуацию, когда службу необходимо гарантированно вывести из строя (например, на время аудита безопасности). Заблокируйте службу Nginx:

```
sudo systemctl mask nginx
```

2. Попробуйте вручную запустить заблокированную службу: `sudo systemctl start nginx`. Зафиксируйте текст системной ошибки отказа в доступе (Failed to start nginx.service: Unit nginx.service is masked.).

3. Разблокируйте службу, вернув ей стандартное поведение:

```
sudo systemctl unmask nginx
```

Этап 5. Поиск ошибок и аудит логов с помощью journalctl

1. Намеренно сломайте конфигурацию веб-сервера. Откройте файл конфигурации `/etc/nginx/nginx.conf` через `sudo nano` и удалите в случайном месте важный символ (например, точку с запятой ; в конце любой строки).
2. Попробуйте перезапустить службу: `sudo systemctl restart nginx`. Система выдаст статус сбоя (failed).
3. Проведите диагностику инцидента. Используя утилиту `journalctl`, выведите логи **строго для этого юнита** за текущую загрузку системы:

```
sudo journalctl -b -u nginx.service
```

4. Найдите строку с ошибкой (строки со сбоями подсвечиваются красным цветом), содержащую отчет встроенного парсера (например, `nginx: [emerg] invalid number of arguments...`). Скопируйте эту строку для отчета.
5. Вернитесь в конфигурационный файл, исправьте ошибку (верните точку с запятой), выполните `sudo systemctl daemon-reload` и успешно запустите службу. Проверьте статус (active).

Отчет должен содержать:

1. **Скриншот №1 (Этап 1):** Вывод команды `sudo systemctl status nginx` в момент, когда служба успешно запущена (должна четко читаться зеленая строка `active (running)` и номер PID).
2. **Скриншот №2 (Этап 2):** Процесс отключения и повторного включения автозагрузки (`disable / enable`), где в терминале видны строки создания/удаления симлинков (`Removed /etc/systemd/system/...`).
3. **Скриншот №3 (Этап 4):** Вывод терминала при попытке старта заблокированной службы после вызова команды `mask`, подтверждающий успешную системную изоляцию юнита.
4. **Скриншот №4 (Этап 5):** Вывод терминала с ошибкой команды `systemctl restart nginx` после того, как конфигурация сервера была намеренно повреждена.
5. **Скриншот №5 (Этап 5):** Окно просмотра журнала `journalctl -b -u nginx.service`, где на экране четко видна строка с техническим

описанием синтаксической ошибки конфигурации, подсвеченная красным цветом.

Контрольные вопросы:

1. Что представляет собой «служба» (демон) в операционной системе Linux, и в чем заключается её принципиальное отличие от обычной пользовательской команды, запущенной в shell-сессии?
2. Почему главный менеджер системных служб systemd всегда имеет в операционной системе идентификатор PID 1? Какую роль он играет в процессе инициализации ОС?
3. Что такое unit-файл в концепции systemd? Где в структуре файловой системы хранятся стандартные unit-файлы, предоставляемые разработчиками ПО, а где — пользовательские конфигурации администратора?
4. Подробно объясните технологическую разницу между действиями `systemctl start` и `systemctl enable`. Может ли служба иметь статус `enabled`, но при этом находиться в состоянии `inactive (dead)`? Приведите пример.
5. В каких сценариях администрирования необходимо применять команду `systemctl reload`, а в каких — `systemctl restart`? Какое из этих действий является более безопасным для работающего продакшн-сервера и почему?
6. Для чего используется команда `systemctl daemon-reload`? Что произойдет, если администратор изменит unit-файл на диске, но забудет выполнить эту команду?
7. Как работает механизм маскирования служб (`systemctl mask`)? Каким образом systemd блокирует юнит на уровне файловой системы, и чем это отличается от стандартного `systemctl disable`?
8. Какие преимущества предоставляет утилита `journalctl` по сравнению с классическим чтением текстовых файлов логов через `tail -f /var/log/...`? Перечислите 3 флага фильтрации журналов systemd, которые чаще всего помогают локализовать сбой.